

- SECURITY TESTING ORCHESTRATION FRAMEWORK

Install it, point it at a target, read the report.

STOF is a command-line security scanner: it logs into a web application as a real browser, maps every page and form, then runs a battery of authentication, session, injection, and access-control checks against it — producing a plain-English, screenshot-backed report anyone can follow. This guide covers setup on both Windows and Linux, first-run configuration, and reading the output.

Requires Python 3.11+ **Interface** command line **Platforms** Windows · Linux · macOS

BEFORE YOU START

What you need

One thing to install, one thing to know before your first scan.

Python 3.11 or newer. STOF is a Python CLI tool — nothing else needs installing by hand. When you first run `stof scan`, it automatically installs its own Python dependencies and downloads its browser engine (a one-time ~200 MB download) if either is missing.

STEP 1

Install STOF

Same four commands on both platforms — create a virtual environment, activate it, install STOF into it. Run these from the folder where you've placed (or cloned) the STOF project.

Windows (PowerShell)

```
PowerShell

# 1. Create a virtual environment
PS> python -m venv .venv

# 2. Activate it
PS> .venv\Scripts\Activate.ps1

# 3. Install STOF and its dependencies
PS> pip install -e ".[dev]"

# 4. Confirm it's on your PATH
PS> stof --help
```

Note. If `Activate.ps1` is blocked by execution policy, run PowerShell as your normal user and execute `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, then retry.

Command Prompt users: activate with `.venv\Scripts\activate.bat` instead of step 2.

Linux / macOS (bash)

```
bash

# 1. Create a virtual environment
$ python3 -m venv .venv

# 2. Activate it
$ source .venv/bin/activate

# 3. Install STOF and its dependencies
$ pip install -e ".[dev]"

# 4. Confirm it's on your PATH
$ stof --help
```

Note. On Debian/Ubuntu, Playwright's browser download needs a few system libraries — if the browser install step fails, run `python -m playwright install-deps` once (may need `sudo`).

STEP 2

Point it at a target

Run the interactive setup wizard once per target. It asks for the target's URL and login credentials, then writes the three files STOF reads on every run — you never have to hand-edit JSON.

```
$ stof configure
```

```
Target base URL: https://demo.testfire.net
Login page URL:  https://demo.testfire.net/login.jsp
Admin username: admin
Admin password: *****
Normal-user username: jsmith
Normal-user password: *****
Modules to enable [idor_tests, auth_tests, xss_tests, ...]: all
```

- ✓ Wrote config/config.json
- ✓ Wrote config/users.json
- ✓ Wrote .env (passwords only live here, never in the JSON files)

Why passwords go in `.env`, not the config files. `config/config.json` and `config/users.json` only ever store a `{{env:VAR}}` token — the real password lives in `.env` alone, which is excluded from version control. Never commit `.env`.

Re-run `stof configure` any time to point at a different target or rotate credentials — your browser and reporting settings are kept as-is.

STEP 3

Run your first scan

One command runs the entire pipeline: crawl the target as every configured role, run every enabled module, and write reports. This is the command you'll use day to day.

```
$ stof scan
```

STOF	·	Security Testing Orchestration Framework
Scan ID		739dcfe5
Target		https://demo.testfire.net
Started		2026-08-27 06:02:55 UTC
Modules		auth_tests, idor_tests, sqli_tests, ...

► CRAWL & ENDPOINT DISCOVERY

```
[CRAWL] Crawling https://demo.testfire.net as role(s) admin, normal...
[AUTH] authenticated user 'admin-01' (role=admin) via form_login
[CRAWL] submitted form #1 on '../feedback.jsp' -> landed on '../feedback.jsp'
[CRAWL] crawl of 'https://demo.testfire.net': visited 54 page(s), found 68
endpoint(s)
```

► VULNERABILITY TESTING

```
running idor_tests... running sqli_tests... running xss_tests...
```

```
(each module prints PASS / FAIL / SKIP per technique as it runs)
```

1 The browser runs headless by default

Nothing visibly opens — pass `--headed` to watch it drive a real browser window instead, useful the first time you point it at a new target.

2 A scan takes a few minutes

Crawling and every module together typically finish in 3–6 minutes against a target of this size — longer targets with more pages/forms take proportionally longer.

3 Write-verb checks are off by default

A small number of techniques (CSRF, some session tests) submit a real request to prove a finding. They're gated behind a config flag you arm deliberately — see [Troubleshooting](#).

STEP 4

Read the results

Every scan writes four report files into `data/reports/`, named after that scan's ID. Pick the one that fits who's reading it.

FILE	WHO IT'S FOR	WHAT'S IN IT
scan_<id>_walkthrough.html	Anyone, non-technical	One page per finding: real browser screenshots and plain-English steps to reproduce it yourself.
scan_<id>.html	Security/engineering	Full technical report — every technique run, request/response evidence, severity, remediation.
scan_<id>.xlsx	Tracking & triage	The same findings as a spreadsheet, ready to filter, assign, and track to closure.
scan_<id>.json	Tooling / CI	Machine-readable findings for feeding into another system.

The screenshot shows the AltoroMutual website interface. At the top, there's a navigation bar with links: [Sign Off](#), [Contact Us](#), [Feedback](#), and a search bar. Below the navigation bar, there's a banner with a green background and a photo of a woman. The main content area is divided into sections: **MY ACCOUNT** (with links like [View Account Summary](#), [View Recent Transactions](#), [Transfer Funds](#), [Search News Articles](#), [Customize Site Language](#)), **PERSONAL** (with a **Hello Admin User** message and a **View Account Details** section showing '800000 Corporate' and a **GO** button), **SMALL BUSINESS**, and **INSIDE ALTORO MUTUAL**. A **DEMO SITE ONLY** banner is visible on the right. The footer contains a disclaimer: 'The AltoroJ website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real banking site. Similarities, if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to <https://www.hcl-software.com/aposcanj/>.' and copyright information: 'Copyright © 2008, 2017, IBM Corporation, All rights reserved. Copyright © 2017, 2026, HCL Technologies, Ltd., All rights reserved.'

From an actual walkthrough report: a SQL injection payload typed into the real login form, then submitted — the very next screenshot shows the application authenticating the attacker as an administrator with no valid password. No Burp Suite request/response dump, just what happened in the browser.

REFERENCE

Commands you'll use

`stof scan` covers most day-to-day use. These break the pipeline into its individual steps when you need finer control.

COMMAND	WHAT IT DOES
<code>stof configure</code>	Interactive wizard — set or change the target and credentials.
<code>stof scan</code>	Full pipeline: crawl, then every enabled module, then all four reports.
<code>stof crawl</code>	Discovery only — maps the target's pages/forms/APIs into <code>data/endpoints.json</code> , no vulnerability checks.
<code>stof test --module idor_tests</code>	Re-run one or more specific modules against the last crawl, without re-crawling. Comma-separate for several.
<code>stof scan --headed</code>	Same as a full scan, but shows the actual browser window while it works.
<code>stof scan --module xss_tests,sqli_tests</code>	Full scan, but only these modules (default: everything enabled in <code>config.json</code>).
<code>stof --help</code>	Full option reference for every command above.

WHEN SOMETHING DOESN'T WORK

Troubleshooting

1 Browser download fails or times out on first run

Re-run `stof scan` — the download resumes. On Linux, missing system libraries show up as a clear error naming `playwright install-deps`; run that once and retry.

2 A scan fails with a network error mid-crawl

Flaky connections to the target occasionally interrupt a scan (a `net::ERR_NETWORK_CHANGED` / `ERR_CONNECTION_REFUSED` -style error in the traceback). This is the target's network, not STOF — just re-run `stof scan`.

3 CSRF / some findings show as SKIP, not FAIL

A handful of techniques submit a real state-changing request to prove a finding, so they're off by default. Set `testing.allow_state_changing_probes` to `true` in `config/config.json` immediately before a scan where you want them to run — it resets itself back to `false` automatically once that scan finishes, so you re-arm it each time on purpose.

4 PowerShell won't run the activation script

Windows blocks unsigned scripts by default. Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once in an admin-free PowerShell window, then retry `.venv\Scripts\Activate.ps1`.

Only scan targets you're authorized to test. STOF sends real requests and, when armed, real write-verb probes — point it only at systems you own or have explicit written permission to assess.